



Lecture 1 — Course Introduction and Claude Code

AI-Assisted Software Development and Intelligent Applications

Note: Sit next to your project 1 partner.

Today's Agenda

1. Syllabus & course overview
2. Claude Code: what it is and how to use it
3. Git & GitLab basics
4. **Activity** – Development Process: Clone, Configure, and Meet the Pipeline

Pre-Class Checklist — Start the Quiz

Before we start — did you complete the setup?

- Installed **Git**, **VS Code**, and the **Claude Code** VS Code plugin
- Installed **Node.js 20**, **Python 3.12**, and **Docker Desktop**
- Verified your account at `git.doit.wisc.edu` (UW GitLab)
- Read the **syllabus**
- Read **Pro Git**, **Chapters 1, 2, and 3**

Having troubles with the software? Please come to office hours to get help.

What Is This Course?

Three units, each building on the last:

Unit 1 – AI-Assisted Software Development *(Weeks 1–5)*

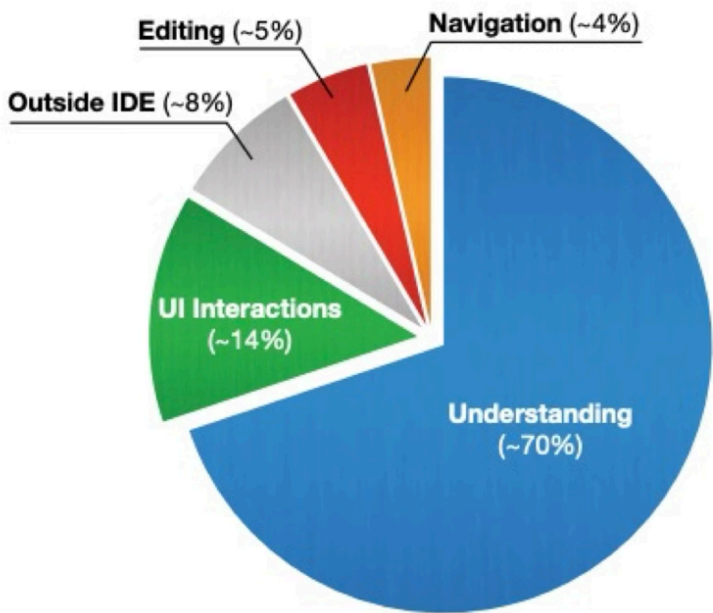
Use AI as a tool across the full dev lifecycle. Learn the full-stack.

Unit 2 – Smart Applications *(Weeks 6–10)*

Build applications with AI as a core component — APIs, RAG, agents.

Unit 3 – Security, Ethics & Harness Engineering *(Weeks 11–15)*

Make AI-integrated systems safe, reliable, and production-ready.



Minelli et al. “I Know What You Did Last Summer: An Investigation of How Developers Spend Their Time” ICPC ’15.

Project	Comprehension	Navigation	Editing	Others
Average	57.62%	23.96%	5.02%	13.40%

Xia et al. “Measuring Program Comprehension: A Large-Scale Field Study with Professionals.” IEEE Trans. Softw. Eng, 2018.

Course Goals

By the end of the semester, you will be able to:

- **Apply universal software-engineering patterns** — skills that hold no matter the tooling
- **Understand coding harnesses** — what they are and how to build your own ([pi.dev](#))
- **Critically evaluate AI companies' claims** about model quality ([Anthropic](#) · [Google](#))
- **Build systems for the intelligence era** — where the tool truly interacts with the user ([Runway Solaris](#))

Course Structure

- **Tuesday & Thursday lectures** — 75 minutes each
- **Flipped classroom** — readings/videos before class; lecture time is for hands-on work
- **5 projects** — brownfield P1–P4 plus a greenfield Final Project
- **Version control** — all work submitted via GitLab, CI/CD pipelines, auto-grade projects
- **AI tooling** — Claude Code is the primary coding assistant; alternative harnesses are okay too

Grading Breakdown

Category	Weight
Projects (P1–P4 + Final)	54%
Exams (3 × 10%)	30%
Participation — attendance + in-class activities	8%
Quizzes	7%
Surveys	1%

Quizzes are low-stakes and test pre-class work.
Will drop your lowest 5 quizzes and 5 activities scores.

The Projects

Project	Weight	Due
P1 — Frontend Side-Scroller Game	8%	Sep 15
P2 — Full-Stack Quiz Game	11%	Oct 6
P3 — RAG Smart Q&A App	11%	Oct 27
P4 — AI Code Review Agent	11%	Nov 17
Final Project	13%	Dec 8

P1 released **yesterday**.

The Exams

Three unit exams — **each covers mainly its unit's content.**

Exam	Date	Covers
Midterm 1	Wed Oct 14, evening	Lectures 1–9 (Unit 1)
Midterm 2	Wed Nov 18, evening	Lectures 10–19 (Unit 2)
Final	Fri Dec 11, 5:05–7:05 PM	Lectures 20–27 (Unit 3)

Traditional written exams — **no computer, closed-book, closed-AI.** One 8.5"×11" note sheet allowed.

Buzzer Checkpoint

**Launch Question 6 – discuss answers, then
continue**

AI Use Policy

Claude Code is your primary development tool in this course — use it.

What is encouraged:

- Using Claude Code to write, debug, refactor, and test code
- Asking Claude to explain unfamiliar code or suggest approaches
- Iterating on Claude's output and making it your own
- Other AI tools — allowed, but you must share those chats manually (Claude Code sessions are auto-logged)

AI Use Policy (cont.)

What is not allowed:

- Submitting AI-generated work you cannot explain
- Letting AI make the decisions without you understanding the result
- Using AI on quizzes or exams — that is academic misconduct

The goal is to make you a better engineer, not to make Claude a better engineer.

Buzzer Checkpoint

**Launch Question 7 – discuss answers, then
continue**

The Tech Stack

Layer	Technology
Frontend	TypeScript + React
Backend	Python + FastAPI
Database	MySQL + Redis
Containers	Docker + Docker Compose
Version Control	Git & GitLab (git.doit.wisc.edu)
CI/CD	GitLab CI/CD pipelines
AI Coding Assistant	Claude Code (CLI, VS Code, or Claude App)

What Is Claude Code?

Two things people conflate:

Claude is a **vision-language model (VLM)** — text and images in, text out. On its own it has no access to your files.

Claude Code is a **harness** — it wraps Claude (or other models) in an agentic loop to read, understand, and edit code and other digital artifacts: it explores your codebase, runs commands, edits files, and checks its own work.

Model = the brain. Harness = the eyes and hands. In Unit 3, you'll build your own.

Where Claude Struggles, You Step In

It struggles with...	You, the operator...
Ambiguity — guesses your intent	state goals, constraints, context
Correctness — no guarantees	run it, test it, read the diff
Forest vs. trees — design drift	own the architecture
Self-awareness — what went wrong?	diagnose and redirect
Higher-order thinking	bring the judgment

Each row is where **you** step in to improve the quality of the result.

One Harness, Three Ways to Run It

Interface	Emphasizes	Your experience as operator
Claude Code CLI	Simplicity	One thread you follow, a few fancy tasks. You orchestrate multiple sessions and review diffs in the chat history yourself.
VS Code + Claude Code	Your codebase	More context on how your code was edited; direct file-based changes more fluidly.
Claude App	Productivity	Easier access to powerful utilities – worktrees, multi-agents, scheduled agents.

All three accomplish the same task – they differ in how *you* experience the harness.

Suggested: start with **CLI or VS Code**; by the end of the course, the **App**.

Claude Code: Core Workflow

Open Claude Code in your interface of choice — it reads your project files for context automatically.

- *"Explain how the game loop in `engine.ts` works"*
- *"Add a jump animation to the player class"*
- Use `@filename` for precise context; `/help` to see commands

Before accepting: Read the diff. Never click Accept without reviewing.

Claude Code: When to Verify

Claude Code is confident even when wrong. Your job is to stay skeptical.

Always verify:

- Output involves a library or API you have not used before
- The change is larger than a few lines
- Claude says "this is the best way" without explaining why
- Tests pass but you do not understand *why* they pass

Buzzer Checkpoint

**Launch Question 8 — discuss answers, then
continue**

Git: Why Version Control?

Without Git, projects look like this:

```
project_final.py  
project_final_v2.py  
project_final_REAL.py  
project_final_REAL_fixed.py
```

With Git: **complete history** of every change, who made it, and when — go back to any point, work on features in parallel. In this course **Git is not optional**: it is how you submit work, how projects are graded, and how the CI/CD pipeline runs.

Git: Core Workflow

```
# Get a copy of a repo  
git clone git@git.doit.wisc.edu:your-username/repo-name.git  
# For Projects, always work on a branch, not main  
git checkout -b feature/my-change  
# Stage and commit your changes  
git add path/to/changed/file.py  
git commit -m "Add jump animation to player"  
# Push to GitLab  
git push -u origin feature/my-change
```

Commit messages: describe the change, not your actions.

"Fix collision at screen edge" not "fixed stuff".

Merging a Branch Locally

When `main` is not protected (e.g., your lectures repo):

```
# After committing on your feature branch:  
git checkout main  
git merge feature/my-change  
git push
```

Use this after finishing every lecture activity, or just do the work on the main branch.

GitLab: Merge Requests

An MR is how team code enters `main` when it is protected — after review and CI:

1. Push your branch
2. Open an MR on GitLab — describe what changed and why
3. A teammate reviews and approves (**not yourself**)
4. CI/CD pipeline runs: tests, lint, format — **this auto-checks your projects**
5. Pipeline green + approved → merge into `main`

Buzzer Checkpoint

**Launch Question 9 – discuss answers, then
continue**

Fork of Upstream Remotes

Your lectures repo is a **fork** of an upstream repo — your copy lives at `git.doit.wisc.edu/cdis/cs/courses/aicoding/fa26/student-repos/lecture/lectures-<your-netid>`

I'll update the upstream repo. You pull new lecture files before each class:

```
# Before each class  
git fetch upstream  
git merge upstream/main
```

⚠ Open Each Lecture's Folder, Not the Repo Root

Claude Code loads `.claude/settings.json`, `.claude/rules/`, and `.claude/skills/` only from the **exact folder it's launched from** — it does not walk up to find the repo root.

Every lecture, open VS Code at **that lecture's folder**, not the repo root:

```
code "Lecture 3 - Claude Code for Brownfield Development/project"
```

Not `code .` from your fork's top. Starts mattering in Lecture 3.

Buzzer Checkpoint

**Launch Question 10 – discuss answers, then
continue**

Activity — Getting Started

Part 1 — Lectures repo:

1. Clone your lectures repo
2. Open `project/README.md` and follow the instructions
3. Commit and push

Part 2 — Project 1:

1. Clone P1 repo
2. Make small change in feature branch, commit, and push
3. Create MR, get it approved — merge stays greyed out while CI is red (expected until P1 Task 4)

Feedback Wanted

How's the pace?

Are there things you want more focus on – topics or techniques you want covered?

Tell me after class, in office hours, or by email – details on the syllabus.



Questions?

Next lecture: Claude Code in Practice — read TypeScript from Scratch and the p5.js Get Started tutorial before class